Verilator: The Verilog to C++/SystemC Compiler

PRIYA RAVINDRAN

One of the biggest questions in a Verilog designer's mind is, "How would it be if the Verilog code can include synthesisable constructs?" A lot of issues that a verification engineer might come across can probably be solved by a simple display statement. We are not talking about the test-bench here but, simply, synthesisable Verilog, with some Property Specification Language (PSL), System Verilog and synthesis assertions thrown in.

If you could just use a single tool, use all of the above language constructs and end up with a SystemC or C++ code that does exactly what you intended the code to do, would that not be truly wonderful? Welcome to Verilator!

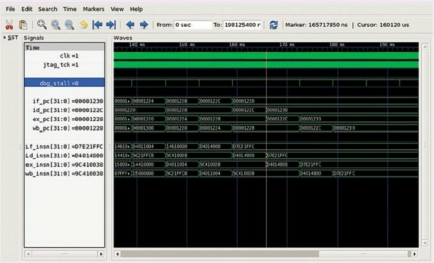

Fig. 1: A tutorial for cycle-accurate SystemC model creation and optimisation using Verilator, by Jeremy Bennett of Embecosm

Verilog simulation with a difference

Verilator is a free Verilog hardware description language (HDL) simulator. It is not simply an alternative for NC-Verilog, Verilog Compiler Simulator (VCS) or even Icarus Verilog. Instead, Verilator is your path to migrate synthesisable Verilog (not behavioural) code into registers and wires on your chip. The output you get is C++ or SystemC that mimics Verilog, plus the synthesisable constructs. Of course, you might need to add a touch of C or Makefile to finally get your code running. Take the files generated from Verilator, add a C++ wrapper file instantiating the top-level module, pass this onto Command Prompt and you are good to go.

The job of Verilator is not to simply translate Verilog to C++ or SystemC directly. Instead, it compiles your code into a much faster model. This highly-optimised version is wrapped inside a C++ or SystemC module. This combination of compiled Verilog and SystemC is about one hundred times faster than interpreted Verilog simulators and ten times faster than standalone SystemC.

Supporting all the right necessities. The fact that Verilator is free for anyone to use makes it an instant favourite. With support provided even for System Verilog assertions and coverage analysis, thorough analysis of your design is many steps easier. If you are working on a large project that requires fast simulations, this should be the tool of your choice—many multi-million gate designs with thousands of modules have been successfully implemented with Verilator. You might want to try that embedded software CPU you have been working on, trying to create an executable model with Verilator, soon.

As fast as it can get

The owners claim that Verilator is the fastest free Verilog HDL simulator and that it simulates faster than even most commercial ones. You might wonder how Verilator manages to run as fast as it does. The answer is that Verilator is not Verilog-compliant! Sounds a bit confusing, does it not? The question is 'How?'

Authors of Verilator explain that most simulators are Verilog-compliant, in the sense that these are event-driven. This makes these wait and follow a sequence, not allowing these to reorder blocks or make netlist-style optimisations. But this, the team believes, is where you stand to gain. It might seem against the natural order, but